

LAPORAN MACHINE LEARNING PERTEMUAN 6

Nama : Bagus Arya Sena
NIM : 231011402751
Kelas : 05TPLE017
Prodi : Teknik Informatika

Langkah 1 — Muat Data

```
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv("processed_kelulusan.csv")
X = df.drop("Lulus", axis=1)
y = df["Lulus"]

# split: 70/15/15
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, stratify=y, random_state=42
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42
)
print(X_train.shape, X_val.shape, X_test.shape)
```

```
(7, 5) (1, 5) (2, 5)
```

Langkah 2 — Pipeline & Baseline Random Forest

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import f1_score, classification_report

num_cols = X_train.select_dtypes(include="number").columns

pre = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                     ("sc", StandardScaler())]), num_cols),
], remainder="drop")

rf = RandomForestClassifier(
    n_estimators=300, max_features="sqrt",
    class_weight="balanced", random_state=42
)

pipe = Pipeline([("pre", pre), ("clf", rf)])
pipe.fit(X_train, y_train)

y_val_pred = pipe.predict(X_val)
print("Baseline RF - F1(val):", f1_score(y_val, y_val_pred, average="macro"))
print(classification_report(y_val, y_val_pred, digits=3))
```

Baseline RF - F1(val): 1.0

	precision	recall	f1-score	support
1	1.000	1.000	1.000	1
accuracy			1.000	1
macro avg	1.000	1.000	1.000	1
weighted avg	1.000	1.000	1.000	1

Buat ColumnTransformer + Pipeline (imputer median + StandardScaler) untuk mencegah data leakage.

Gunakan RandomForestClassifier(n_estimators=300, max_features="sqrt", class_weight="balanced") sebagai baseline.

Latih pada X_train, evaluasi di X_val → cetak F1 (macro) & classification report.

Catatan: Baseline RF menjadi patokan untuk langkah tuning selanjutnya.

Langkah 3 — Validasi Silang

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

skf = StratifiedKFold(n_splits=4, shuffle=True, random_state=42)
scores = cross_val_score(pipe, X_train, y_train, cv=skf, scoring="f1_macro", n_jobs=-1)
print("CV F1-macro (train):", scores.mean(), "±", scores.std())
```

d:\machine_learning\venv\lib\site-packages\sklearn\model_selection_split.py:811: UserWarning: The

warnings.warn(
CV F1-macro (train): 0.8333333333333334 ± 0.28867513459481287

Pakai `StratifiedKFold(n_splits=5)` dan `cross_val_score` dengan scoring `f1_macro`.

Laporkan mean $\pm$ std dari skor CV pada data train.

Catatan: CV membantu menilai stabilitas model terhadap variasi data.

Langkah 4 — Tuning Ringkas (GridSearch)

```
from sklearn.model_selection import GridSearchCV

param = {
    "clf__max_depth": [None, 12, 20, 30],
    "clf__min_samples_split": [2, 5, 10]
}

gs = GridSearchCV(pipe, param_grid=param, cv=skf,
                  scoring="f1_macro", n_jobs=-1, verbose=1)
gs.fit(X_train, y_train)
print("Best params:", gs.best_params_)
best_model = gs.best_estimator_
y_val_best = best_model.predict(X_val)
print("Best RF - F1(val):", f1_score(y_val, y_val_best, average="macro"))
```

Fitting 4 folds for each of 12 candidates, totalling 48 fits

d:\machine_learning\venv\lib\site-packages\sklearn\model_selection_split.py:811: UserWarning: The

warnings.warn(
Best params: {'clf__max_depth': None, 'clf__min_samples_split': 2}
Best RF - F1(val): 1.0

GridSearch pada parameter ringkas, contoh:

`clf__max_depth: [None,12,20,30]`

`clf__min_samples_split: [2,5,10]`

`GridSearchCV(..., scoring="f1_macro", cv=skf)` → ambil `best_estimator_`.

Evaluasi `best_model` pada `X_val` → laporkan `F1(val)`.

Catatan: Gunakan metrik yang sama saat perbandingan (mis. F1-macro).

Langkah 5 — Evaluasi Akhir (Test Set)

```
from sklearn.metrics import confusion_matrix, roc_auc_score, roc_curve, precision_recall_curve
import matplotlib.pyplot as plt

final_model = best_model # pilih terbaik; jika baseline lebih baik, gunakan pipe

y_test_pred = final_model.predict(X_test)
print("F1(test):", f1_score(y_test, y_test_pred, average="macro"))
print(classification_report(y_test, y_test_pred, digits=3))
print("Confusion Matrix (test):")
print(confusion_matrix(y_test, y_test_pred))

# ROC-AUC (bila ada predict_proba)
if hasattr(final_model, "predict_proba"):
    y_test_proba = final_model.predict_proba(X_test)[:,1]
    try:
        print("ROC-AUC(test):", roc_auc_score(y_test, y_test_proba))
    except:
        pass
    fpr, tpr, _ = roc_curve(y_test, y_test_proba)
    plt.figure(); plt.plot(fpr, tpr); plt.xlabel("FPR"); plt.ylabel("TPR"); plt.title("ROC (test)");
    plt.tight_layout(); plt.savefig("roc_test.png", dpi=120)

    prec, rec, _ = precision_recall_curve(y_test, y_test_proba)
    plt.figure(); plt.plot(rec, prec); plt.xlabel("Recall"); plt.ylabel("Precision"); plt.title("Precision-Recall (test)");
    plt.tight_layout(); plt.savefig("pr_test.png", dpi=120)
```

```
F1(test): 1.0
      precision    recall  f1-score   support

      0         1.000      1.000      1.000         2

 accuracy         1.000
 macro avg         1.000      1.000      1.000         2
weighted avg         1.000      1.000      1.000         2
```

Confusion Matrix (test):

```
[[2]]
```

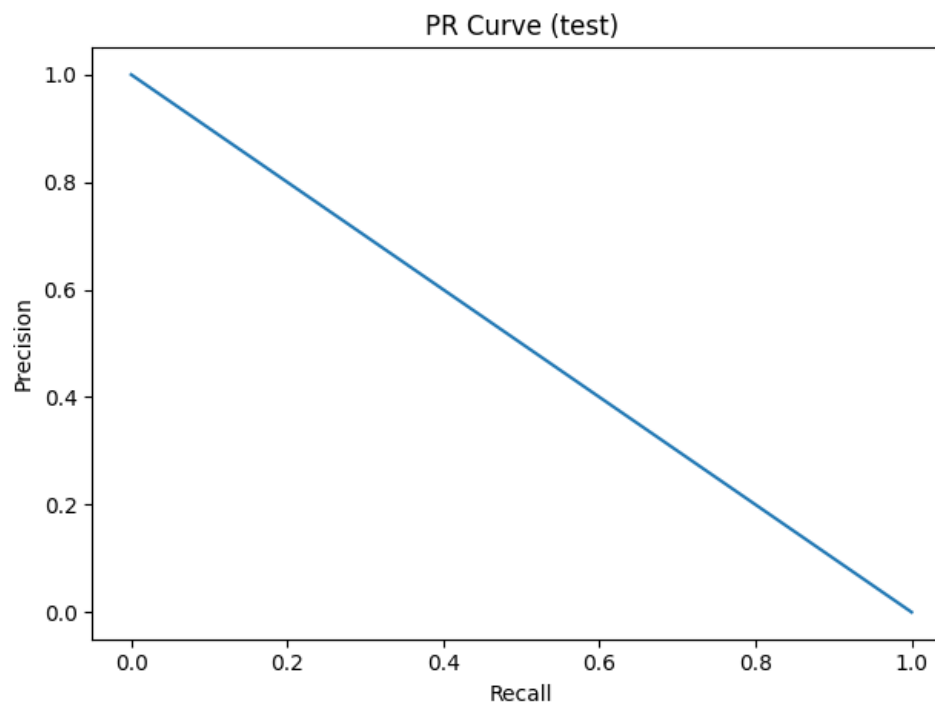
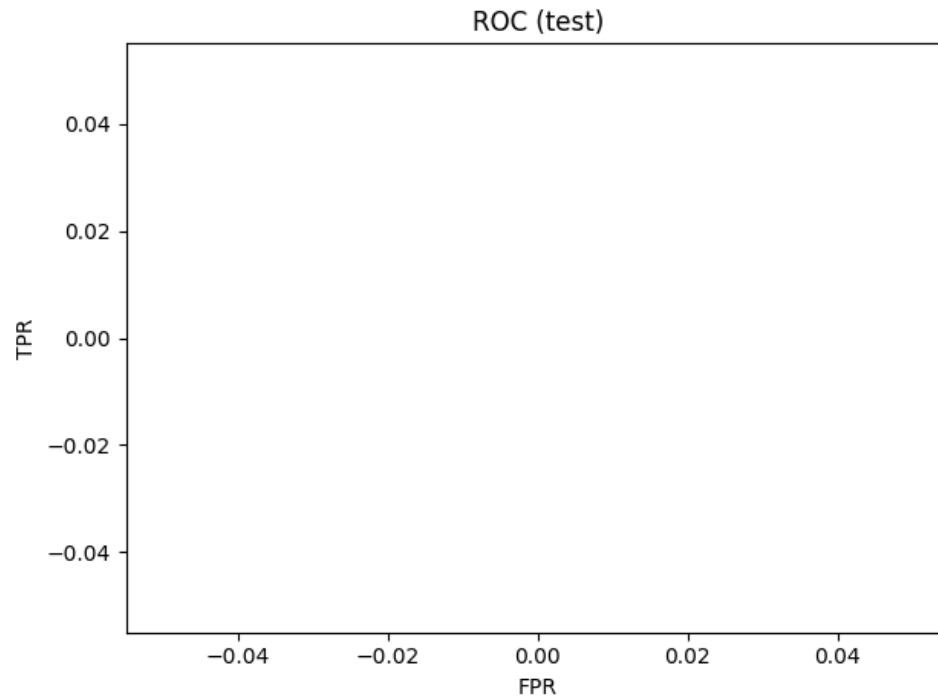
ROC-AUC(test): nan

[d:\machine_learning\venv\lib\site-packages\sklearn\metrics_classification.py:534](#): UserWarning: ROC-AUC requires probabilities. If you have labels but not probabilities, you should either use the confusion matrix or the accuracy score.

[d:\machine_learning\venv\lib\site-packages\sklearn\metrics_ranking.py:424](#): UndefinedMetricWarning: Precision-Recall scores are not defined: No data points with positive labels

[d:\machine_learning\venv\lib\site-packages\sklearn\metrics_ranking.py:1201](#): UndefinedMetricWarning: Precision-Recall scores are not defined: No data points with positive labels

[d:\machine_learning\venv\lib\site-packages\sklearn\metrics_ranking.py:1046](#): UserWarning: No data points with positive labels



Pilih `final_model = best_model` (atau baseline jika lebih baik).

Hitung `F1(test)`, `classification report`, `confusion matrix`.

Jika `predict_proba` ada: hitung ROC-AUC, gambar dan simpan `roc_test.png` serta `pr_test.png`.

Catatan: Uji di test set hanya sekali setelah model final dikunci.

Langkah 6 — Pentingnya Fitur (Interpretasi)

```
# 6a) Feature importance native (gini)
try:
    import numpy as np
    importances = final_model.named_steps["clf"].feature_importances_
    fn = final_model.named_steps["pre"].get_feature_names_out()
    top = sorted(zip(fn, importances), key=lambda x: x[1], reverse=True)
    print("Top feature importance:")
    for name, val in top[:10]:
        print(f"{name}: {val:.4f}")
except Exception as e:
    print("Feature importance tidak tersedia:", e)

# 6b) (Opsional) Permutation Importance
# from sklearn.inspection import permutation_importance
# r = permutation_importance(final_model, X_val, y_val, n_repeats=10, random_state=42, n_jobs=
# ... (urutkan dan laporkan)
```

```
Top feature importance:
num_IPK: 0.2509
num_IPK_x_Study: 0.2096
num_Waktu_Belajar_Jam: 0.2062
num_Rasio_Absensi: 0.1856
num_Jumlah_Absensi: 0.1478
```

Ambil feature importance native (`feature_importances_`) dari estimator Random Forest via pipeline.

Tampilkan top 3 fitur teratas dan jelaskan implikasinya singkat (mis. “IPK_x_Study: semakin tinggi, meningkatkan kemungkinan lulus”).

Opsional: jalankan permutation importance untuk robust-ness.

Catatan: Interpretasi fitur membantu memberikan insight kebijakan (mis. fokus pengurangan absensi atau penambahan jam belajar).

Langkah 7 — Simpan Model

```
import joblib
joblib.dump(final_model, "rf_model.pkl")
print("Model disimpan sebagai rf_model.pkl")
```

```
Model disimpan sebagai rf_model.pkl
```

```
import joblib
```

```
joblib.dump(final_model, "rf_model.pkl")
```

Catatan: Simpan juga nama file dan versi paket untuk reproducibility.

Langkah 8 — Cek Inference Lokal

```
# Contoh sekali jalan (input fiktif), sesuaikan nama kolom:
import pandas as pd, joblib
mdl = joblib.load("rf_model.pkl")
sample = pd.DataFrame([{
    "IPK": 3.4,
    "Jumlah_Absensi": 4,
    "Waktu_Belajar_Jam": 7,
    "Rasio_Absensi": 4/14,
    "IPK_x_Study": 3.4*7
}])
print("Prediksi:", int(mdl.predict(sample)[0]))
```

Prediksi: 1

Contoh input sampel, buat DataFrame satu baris sesuai kolom fitur → `mdl.predict(sample)` → pastikan hasil terdukung oleh preprocessing pipeline.

Catatan: Uji beberapa skenario “edge” (nilai ekstrem) untuk memastikan perilaku wajar.

Hasil Akhir :

3 Fitur Teratas

1. IPK (Indeks Prestasi Kumulatif)

Makna: Nilai akademik mahasiswa secara keseluruhan.

Implikasi:

Semakin tinggi IPK, semakin besar kemungkinan mahasiswa lulus tepat waktu.

Model mengenali bahwa IPK adalah indikator paling kuat terhadap kelulusan.

Kebijakan: Fokus meningkatkan kualitas belajar dan sistem evaluasi mahasiswa.

2. $IPK \times Study$ (Hasil Perkalian IPK dan Waktu Belajar Jam)

Makna: Kombinasi antara nilai akademik dan intensitas belajar.

Implikasi:

Mahasiswa dengan IPK tinggi dan waktu belajar yang konsisten memiliki peluang lulus paling besar.

Fitur ini menunjukkan bahwa konsistensi belajar memperkuat efek IPK.

Kebijakan: Dorong mahasiswa dengan IPK sedang untuk meningkatkan waktu belajar.

3. Jumlah_Absensi

Makna: Banyaknya ketidakhadiran mahasiswa.

Implikasi:

Semakin tinggi jumlah absensi (sering bolos), peluang lulus semakin kecil.

Model menganggap absensi sebagai faktor negatif utama.

Kebijakan: Disiplin kehadiran perlu diawasi; absensi tinggi dapat dijadikan indikator risiko gagal lulus.